Akselerasi Paralel Penyelesaian Sistem Linear Analisis Nodal Termodifikasi (MNA) Menggunakan OpenMP dan OpenCL

Fairuz Apuilla Rahagi*, Adam Mohammad Febrian†, Ahmad Ramadhan‡
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung, Bandung, Indonesia
Email: *13222107@mahasiswa.itb.ac.id, †13222015@mahasiswa.itb.ac.id, ‡13222110@mahasiswa.itb.ac.id

***Abstract*—Pada dasarnya, simulasi rangkaian elektrik dapat direduksi menjadi penyelesaian sistem persamaan linear $A\mathbf{x} = \mathbf{b}$ yang dibangun melalui metode *Modified Nodal Analysis* (MNA). Inti komputasinya, yakni dekomposisi LU pada matriks padat, memiliki kompleksitas $O(n^3)$ dan menjadi penghambat utama (*bottleneck*) ketika ukuran rangkaian membesar. Makalah ini memaparkan rancangan dan implementasi sebuah solver MNA yang menyelesaikan sistem linear tersebut dengan tiga *backend* berbeda: sekuensial satu inti, paralel multi-inti berbasis OpenMP, dan paralel GPU berbasis OpenCL. Ketiganya menggunakan algoritma dekomposisi LU yang identik secara matematis sehingga perbandingan kinerjanya benar-benar setara. Kebenaran solver divalidasi terhadap nilai analitik dan terhadap simulator komersial LTspice. Hasil pengujian (*benchmark*) pada matriks berukuran $n = 51$ hingga $n = 1001$ memperlihatkan adanya fenomena *crossover* yang khas: untuk matriks kecil seluruh backend paralel justru lebih lambat akibat ongkos paralelisasi, dan paralelisasi baru memberikan keuntungan ketika $n \gtrsim 500$. Pada $n = 1001$ diperoleh percepatan tertinggi $2{,}64\times$ (OpenMP 4 utas) dan $2{,}35\times$ (OpenCL, waktu komputasi). Temuan ini menegaskan prinsip dasar komputasi performa tinggi: paralelisasi selalu menuntut ongkos, dan baru menguntungkan apabila volume kerja cukup besar untuk menutupi *overhead* tersebut.**

***Index Terms*—komputasi performa tinggi, analisis nodal termodifikasi, dekomposisi LU, OpenMP, OpenCL, GPU, paralelisasi**

I. Pendahuluan

Penyelesaian sistem persamaan linear merupakan kernel komputasi yang muncul di hampir seluruh cabang sains dan rekayasa: analisis elemen hingga, pemrosesan sinyal, pembelajaran mesin, hingga simulasi rangkaian elektrik. Pada simulasi rangkaian, perangkat lunak kelas SPICE membangun model matematis rangkaian, lalu menyelesaikan sistem $A\mathbf{x} = \mathbf{b}$ untuk memperoleh tegangan tiap simpul dan arus tiap sumber [1]. Ketika jumlah simpul bertambah, biaya penyelesaian sistem inilah yang mendominasi keseluruhan waktu simulasi.

Mata kuliah Komputasi Performa Tinggi menekankan bagaimana sebuah persoalan komputasi yang *compute-bound* dapat dipercepat dengan memanfaatkan paralelisme perangkat keras modern, baik berupa banyak inti CPU maupun ribuan inti GPU. Atas dasar itu, tim memilih persoalan penyelesaian sistem linear MNA sebagai studi kasus, karena: (i) persoalan ini nyata dan dapat diverifikasi terhadap perangkat lunak komersial; (ii) kernel komputasinya, dekomposisi LU padat, adalah *benchmark* paralel kanonik yang sama dengan yang dipakai LINPACK untuk memeringkat superkomputer dunia [4]; dan (iii) persoalan ini bukan persoalan pengurutan (*sorting*), sesuai batasan tugas.

Kontribusi makalah ini adalah: (1) perumusan dan implementasi pembangun matriks MNA lengkap dengan mekanisme *stamping*; (2) tiga implementasi solver LU yang identik secara algoritmik namun berbeda strategi paralelisasinya (sekuensial, OpenMP, OpenCL); (3) kerangka pengujian otomatis yang memvalidasi kebenaran dan mengukur kinerja; serta (4) analisis menyeluruh atas perilaku percepatan, termasuk fenomena *crossover* dan keterbatasan skalabilitas.

II. Permasalahan Komputasi

A. Rumusan Persoalan

Diberikan sebuah rangkaian linear yang terdiri atas resistor, sumber tegangan, dan sumber arus, persoalannya adalah mencari potensial tiap simpul (relatif terhadap simpul referensi/ground) dan arus cabang tiap sumber tegangan. Dengan metode MNA, persoalan fisis ini diubah menjadi sistem linear

$$A\mathbf{x} = \mathbf{b}, \qquad A \in \mathbb{R}^{n\times n},\ \mathbf{x}, \mathbf{b} \in \mathbb{R}^n, \tag{1}$$

dengan $n = (N-1) + M$, di mana N adalah jumlah simpul (termasuk ground) dan M adalah jumlah sumber tegangan. Vektor solusi $\mathbf{x}$ memuat $N-1$ tegangan simpul disusul M arus cabang.

B. Karakteristik Komputasi

Penyelesaian (1) melalui eliminasi Gauss / dekomposisi LU membutuhkan sekitar $\frac{2}{3}n^3$ operasi *floating-point*, sementara substitusi maju-mundur hanya $O(n^2)$. Dengan demikian, untuk n besar, tahap faktorisasi LU mendominasi total komputasi. Sifat $O(n^3)$ inilah yang menjadikan persoalan ini bersifat *compute-bound* sekaligus menjadi target ideal untuk paralelisasi: setiap kali n digandakan, beban kerja meningkat delapan kali lipat. Perlu dicatat bahwa persoalan ini sama sekali tidak melibatkan pengurutan; beban komputasinya murni aljabar linear numerik.

III. Solusi yang Telah Ada

Penyelesaian sistem linear dari analisis rangkaian merupakan persoalan klasik dengan beragam solusi mapan.

TABLE I
TIGA ALTERNATIF STRATEGI PARALELISASI

Backend	Paralelisme	Catatan
Sekuensial	Tidak ada	LU + *partial pivoting* di CPU; menjadi *baseline*.
OpenMP	Multi-inti CPU	`#pragma omp parallel for` pada perulangan eliminasi baris; `schedule(dynamic)`; pivoting tetap sekuensial.
OpenCL	Banyak inti GPU	Dua kernel 1D/2D per kolom pivot; tanpa pivoting; hasil diverifikasi via residu.

Simulator SPICE. Perangkat lunak seperti SPICE [1] dan turunannya (LTspice, ngspice) menggunakan MNA yang dipasangkan dengan pemfaktoran LU jarang (*sparse LU*). Pendekatan ini sangat efisien untuk rangkaian nyata yang matriksnya didominasi nol.

Pustaka aljabar linear padat. LAPACK dan BLAS menyediakan rutin faktorisasi LU padat (mis. `dgetrf`) yang teroptimasi tinggi terhadap hierarki memori melalui algoritma berblok [3]. *Benchmark* LINPACK yang dibangun di atasnya menjadi tolok ukur resmi peringkat TOP500 [4].

Solver jarang. Untuk matriks jarang, pustaka seperti KLU dan SuiteSparse [5] mengeksploitasi struktur nol sehingga menghindari komputasi yang sia-sia. Khusus matriks tridiagonal, algoritma Thomas menyelesaikan sistem dalam $O(n)$.

Akselerasi GPU. Pustaka seperti cuSOLVER dan MAGMA menyediakan faktorisasi LU pada GPU, memanfaatkan ribuan inti untuk mengakselerasi operasi level matriks [7].

Solusi-solusi komersial dan pustaka ini sangat matang, namun bersifat *black box* sehingga kurang sesuai untuk tujuan pembelajaran. Untuk tujuan tugas ini, tim membangun solver dari nol agar dapat mengontrol dan membandingkan strategi paralelisasi secara langsung, dengan LTspice diposisikan sebagai acuan kebenaran (*ground truth*).

IV. ALTERNATIF SOLUSI TIM

Tim mengusulkan tiga implementasi solver yang menyelesaikan persoalan yang sama, (1), dengan algoritma dekomposisi LU yang identik, tetapi berbeda pada strategi pemetaan komputasi ke perangkat keras. Strategi ini diringkas pada Tabel I.

Sekuensial berfungsi sebagai garis dasar yang benar dan stabil secara numerik. **OpenMP** mendistribusikan perulangan eliminasi baris (yang antar-barisnya saling bebas) ke beberapa utas CPU dengan ongkos perubahan kode minimal. **OpenCL** memetakan tiap sel sub-matriks aktif ke satu *work-item* GPU, mengeksploitasi paralelisme data masif namun dengan ongkos transfer data host↔device dan tanpa pivoting (yang sukar diimplementasikan secara efisien di GPU).

V. DESAIN SISTEM

A. Arsitektur Sistem

Sistem dirancang sebagai alur pemrosesan berlapis (Gambar 1): *lapisan masukan* membangun objek rangkaian; *lapisan model* mengubah rangkaian menjadi matriks MNA; *lapisan komputasi* menyelesaikan sistem linear dengan salah satu dari tiga backend; dan *lapisan keluaran* memverifikasi serta melaporkan hasil. Pemisahan ini membuat ketiga solver dapat dipertukarkan tanpa mengubah bagian pembangun matriks.

B. Diagram Aliran Data (DFD)

Diagram konteks (DFD level 0) pada Gambar 2 menempatkan sistem sebagai satu proses tunggal yang berinteraksi dengan dua entitas eksternal: *Pengguna* (yang memberi parameter rangkaian dan menerima hasil) dan *LTspice* (sebagai sumber tegangan referensi untuk validasi). Diagram level 1 pada Gambar 3 memecah sistem menjadi empat proses: (1) membangun rangkaian, (2) menyusun matriks MNA, (3) menyelesaikan sistem linear, dan (4) memverifikasi serta melaporkan hasil; ditambah tiga penyimpanan data: berkas kernel GPU, berkas hasil *benchmark*, dan berkas netlist.

C. Formulasi MNA dan Aturan Stamping

Matriks MNA dibangun melalui mekanisme *stamping*: tiap komponen menyumbangkan kontribusi ke sel-sel tertentu dari A dan $\mathbf{b}$. Untuk resistor dengan konduktansi $G = 1/R$ antara simpul i dan j (indeks digeser karena simpul ground tidak masuk matriks):

$$A_{ii} \mathrel{+}= G, \quad A_{jj} \mathrel{+}= G, \quad A_{ij} \mathrel{-}= G, \quad A_{ji} \mathrel{-}= G. \tag{2}$$

Untuk sumber tegangan ke-k bernilai V_k antara simpul p dan q, ditambahkan satu baris/kolom cabang berindeks $r = (N-1) + k$:

$$A_{pr} = A_{rp} = +1, \quad A_{qr} = A_{rq} = -1, \quad b_r = V_k, \tag{3}$$

sedangkan sumber arus I menyumbang $b_p \mathrel{+}= I$ dan $b_q \mathrel{-}= I$. Potongan inti rutin *stamping* ditunjukkan pada Listing 1.

Listing 1. Inti rutin stamping MNA (`mna.cpp`).

```
double G = 1.0 / r.resistance;
if (i>0) A[i-1][i-1] += G;
if (j>0) A[j-1][j-1] += G;
if (i>0 && j>0) {
  A[i-1][j-1] -= G;
  A[j-1][i-1] -= G;
}
```

D. Algoritma LU dan Diagram Alir

Ketiga backend menjalankan dekomposisi LU yang sama: untuk tiap kolom pivot k, dipilih pivot terbesar (pivoting parsial, hanya pada jalur CPU), baris ditukar, lalu seluruh baris di bawah pivot dieliminasi. Tahap eliminasi inilah yang diparalelkan. Algoritma 1 merangkum prosedurnya, dan Gambar 4 menyajikan diagram alirnya beserta penandaan wilayah paralel.

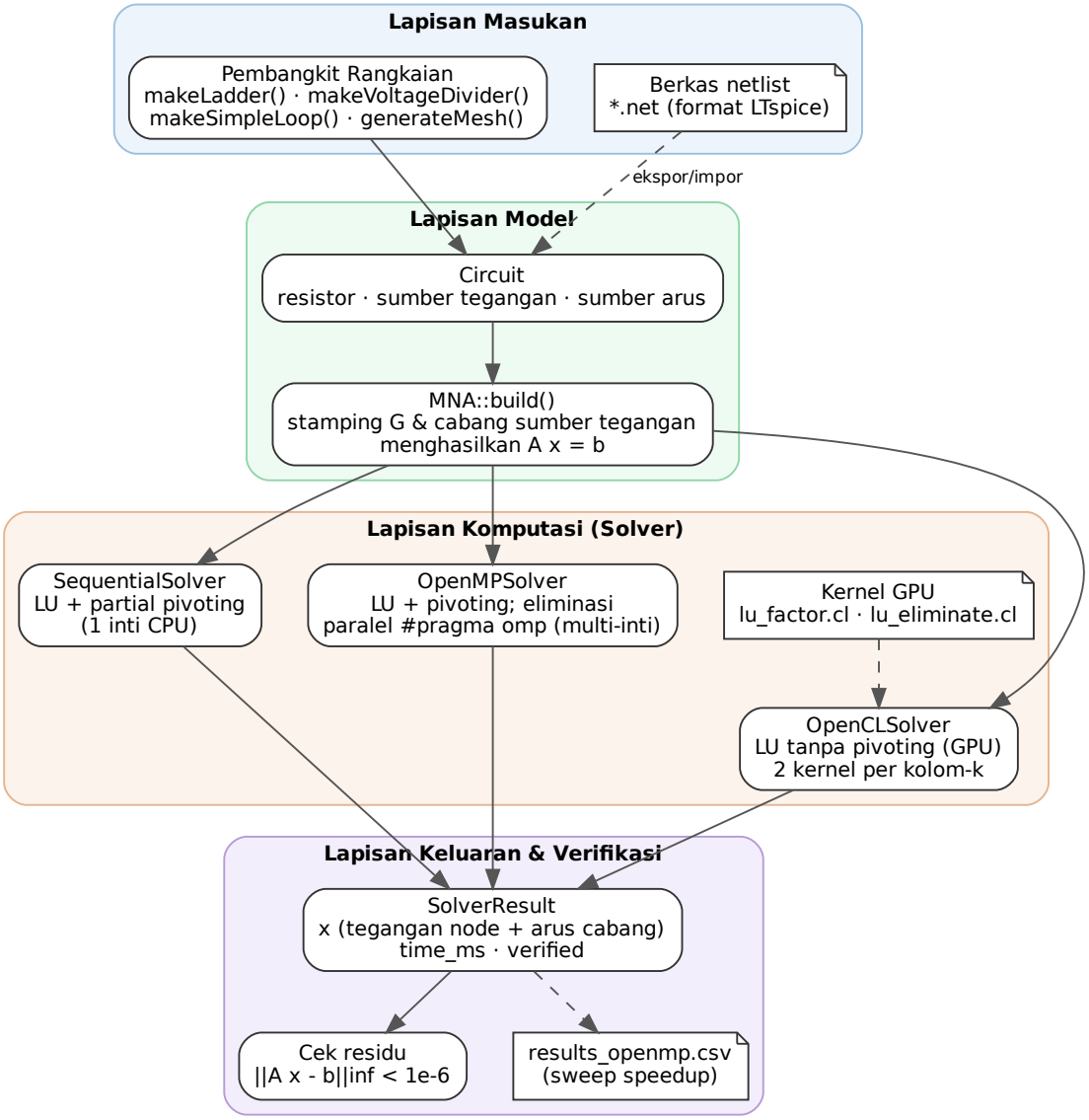


Fig. 1. Arsitektur sistem solver MNA paralel. Pembangun matriks MNA bersifat tunggal, sementara ketiga backend solver dapat dipertukarkan dan menghasilkan struktur `SolverResult` yang seragam.

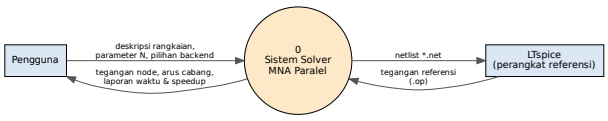


Fig. 2. DFD level 0 (diagram konteks).

VI. Implementasi

A. Lingkungan dan Perkakas

Solver ditulis dalam C++17 dan dikompilasi dengan `g++` ($-O2$). Paralelisme CPU memakai OpenMP (`-fopenmp`), sedangkan paralelisme GPU memakai OpenCL 1.2. Sistem *build* berbasis `Makefile` yang mendeteksi keberadaan SDK OpenCL secara otomatis; bila SDK tidak tersedia, sebuah *stub* ditautkan sehingga program tetap dapat dikompilasi dan solver GPU melaporkan diri sebagai tidak tersedia saat dijalankan. Antarmuka baris perintah menyediakan mode `-test` (vali-

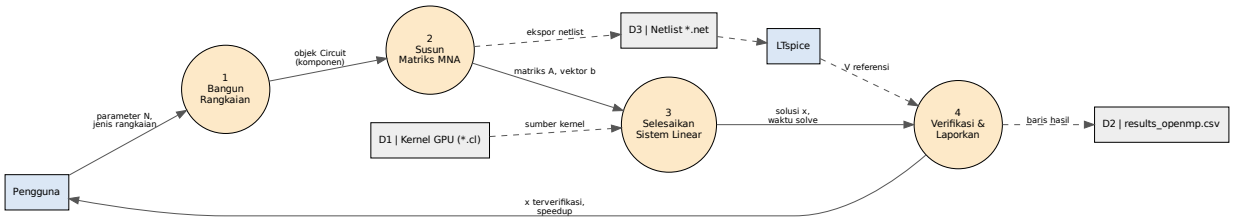


Fig. 3. DFD level 1: empat proses utama beserta aliran data dan penyimpanan data.

Algorithm 1: Dekomposisi LU dengan pivoting parsial

Input: matriks A, vektor $\mathbf{b}$ berukuran n
Output: solusi $\mathbf{x}$

for $k \leftarrow 0$ **to** $n-1$ **do**
  cari $p \geq k$ dengan $|A_{pk}|$ maksimum;
  `// pivoting (CPU)`
  if $|A_{pk}| < 10^{-14}$ **then**
    return *matriks singular*
  tukar baris k dan p;
  `/* -- wilayah paralel (OpenMP / OpenCL) --  */`
  for $i \leftarrow k+1$ **to** $n-1$ **do**
    $f \leftarrow A_{ik}/A_{kk}$; $A_{ik} \leftarrow f$;
    for $j \leftarrow k+1$ **to** $n-1$ **do** $A_{ij} \mathrel{-}= f\,A_{kj}$;
    $b_i \mathrel{-}= f\,b_k$;
substitusi maju lalu mundur untuk memperoleh $\mathbf{x}$;

dasi), `-bench N` (uji kinerja), dan `-compare` (pembandingan terhadap berkas referensi).

B. Backend OpenMP

Implementasi OpenMP menambahkan satu direktif pada perulangan eliminasi baris, sebagaimana Listing 2. Karena iterasi atas indeks baris i saling bebas dalam satu langkah kolom, perulangan ini aman diparalelkan. Pencarian pivot dan penukaran baris tetap dijalankan sekuensial; bagian sekuensial inilah yang membatasi percepatan sesuai hukum Amdahl [8]. Klausa `schedule(dynamic)` dipilih agar beban tetap seimbang ketika sub-matriks menyusut pada iterasi k yang besar.

Listing 2. Wilayah paralel OpenMP (`lu_openmp.cpp`).

```
#pragma omp parallel for schedule(dynamic)
    num_threads(nt)
for (int i = k+1; i < n; ++i) {
  double factor = A[i][k] / A[k][k];
  A[i][k] = factor; // simpan pengali (L)
  for (int j = k+1; j < n; ++j)
    A[i][j] -= factor * A[k][j];
  b[i] -= factor * b[k];
}
```

C. Backend OpenCL

Backend GPU melakukan inisialisasi sekali (memilih perangkat GPU, atau CPU sebagai cadangan, lalu mengompilasi kernel), kemudian menyelesaikan sistem dengan menyalin matriks ke memori GPU dan menjalankan dua kernel untuk tiap kolom pivot k:

- `compute_factors` (kernel 1D, $n-k-1$ *work-item*) menghitung pengali $f_i = A_{ik}/A_{kk}$ dan menyimpannya kembali ke bagian segitiga bawah A sebagai faktor L;
- `lu_eliminate` (kernel 2D, $(n-k-1)^2$ *work-item*) memperbarui setiap sel sub-matriks aktif, satu sel per *work-item*.

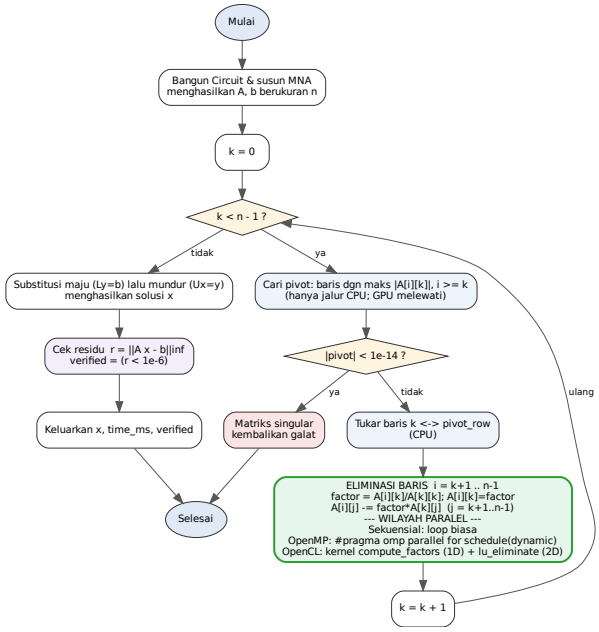


Fig. 4. Diagram alir solver LU. Blok eliminasi baris (kotak hijau) merupakan wilayah paralel yang diimplementasikan berbeda pada tiap backend.

Kernel eliminasi inti ditunjukkan pada Listing 3. Substitusi maju-mundur dilakukan kembali di CPU. Karena jalur GPU tidak melakukan pivoting, setiap hasil diverifikasi melalui residu $\|A\mathbf{x}-\mathbf{b}\|_\infty$; ambang 10^{-6} menandai solusi yang sah.

Listing 3. Kernel eliminasi GPU (`lu_eliminate.cl`).

```
int i = get_global_id(0); // offset baris dari k+1
int j = get_global_id(1); // offset kolom dari k+1
int row = i + k + 1, col = j + k + 1;
```

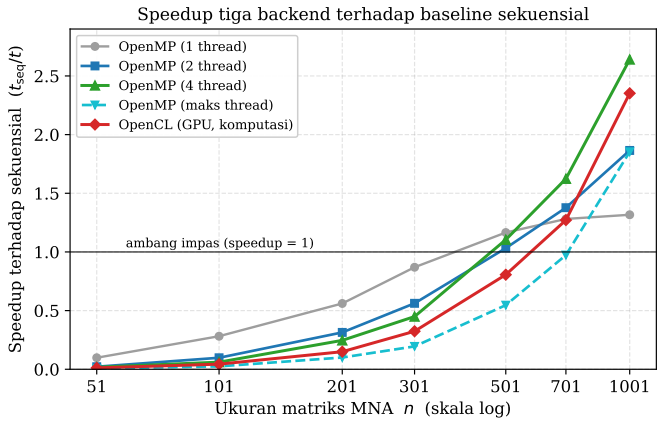


Fig. 5. Kurva percepatan tiga backend. Garis impas (*speedup* = 1) memisahkan wilayah rugi (atas-kiri) dari wilayah untung (kanan).

```
if (row >= n || col >= n) return;
A[row*n + col] -= factors[i] * A[k*n + col];
```

D. Pengukuran Waktu

Waktu eksekusi CPU diukur dengan `std::chrono::high_resolution_clock`, sedangkan waktu komputasi GPU diukur melalui *event profiling* OpenCL beresolusi nanodetik. Untuk GPU dibedakan *waktu komputasi* (hanya kernel) dan *waktu total* (komputasi ditambah transfer data). Tiap konfigurasi dijalankan lima kali dan waktu *minimum* dilaporkan guna meredam fluktuasi akibat penjadwalan sistem operasi.

VII. Hasil dan Analisis

A. Validasi Kebenaran

Suite validasi menguji tiga rangkaian dengan jawaban analitik yang diketahui: pembagi tegangan (1,667 V, 3,333 V, 5 V), loop sederhana ($I = 2$ A), dan tangga 10-tahap. Seluruhnya LULUS. Selain itu, netlist tangga dan *mesh* 3×3 diekspor dan disimulasikan pada LTspice; berkas log menunjukkan iterasi titik kerja (*.op*) berhasil pada kedua topologi (ukuran matriks 7 dan 9). Kesesuaian ini memberi keyakinan bahwa pembangun MNA benar tidak hanya pada rantai sederhana tetapi juga pada topologi dua dimensi.

B. *Hasil* Benchmark

Tabel II merangkum waktu eksekusi dan percepatan untuk n dari 51 hingga 1001, dan Gambar 5 memvisualisasikan kurva percepatannya. Percepatan didefinisikan sebagai $S = t_{\text{seq}}/t$.

C. Analisis

Fenomena *crossover*. Untuk matriks kecil ($n \leq 301$) seluruh backend paralel jauh lebih lambat daripada sekuensial—hingga puluhan kali lipat. Penyebabnya adalah *overhead* paralelisasi: biaya pembuatan dan sinkronisasi utas pada OpenMP, serta biaya transfer data dan peluncuran kernel pada OpenCL, melampaui beban komputasi sesungguhnya yang masih ringan. Ketika n membesar, beban $O(n^3)$ tumbuh jauh lebih cepat daripada *overhead* yang relatif tetap, sehingga paralelisasi mulai menguntungkan di sekitar $n = 500$–700 dan mencapai puncaknya pada $n = 1001$. Inilah pelajaran inti komputasi performa tinggi: *paralelisasi tidak datang tanpa ongkos, dan baru unggul ketika persoalan cukup besar.*

Empat utas mengungguli utas maksimum. Terdapat hasil yang menarik: pada $n = 1001$, OpenMP dengan 4 utas ($2{,}64\times$) justru lebih cepat daripada OpenMP dengan utas maksimum ($1{,}85\times$). Hal ini mengindikasikan bahwa menambah utas melewati titik tertentu malah merugikan, karena tahap eliminasi bersifat *memory-bound*—setiap utas membaca baris matriks secara beruntun dari memori—sehingga utas-utas tambahan justru saling berebut *bandwidth* memori dan menimbulkan *overhead* penjadwalan serta kemungkinan *oversubscription* pada inti logis (*hyperthread*). Temuan ini menegaskan bahwa skalabilitas paralel dibatasi oleh sumber daya bersama, bukan semata jumlah utas.

Ongkos transfer GPU. Percepatan OpenCL berdasarkan waktu total ($2{,}25\times$) sedikit lebih rendah daripada berdasarkan waktu komputasi ($2{,}35\times$) akibat transfer data host↔device. Selisihnya kecil karena faktorisasi $O(n^3)$ jauh mendominasi transfer $O(n^2)$, tetapi tetap memperlihatkan bahwa transfer data merupakan komponen biaya nyata pada komputasi GPU.

Kompromi tanpa pivoting pada GPU. Jalur GPU sengaja melewati pivoting parsial karena operasi penukaran baris sukar diparalelkan efisien di GPU. Untuk rangkaian tangga yang *well-conditioned*, pivoting tidak pernah benar-benar diperlukan sehingga solusi GPU tetap benar—dibuktikan oleh cek residu yang selalu lolos. Cek residu inilah jaring pengaman yang akan menangkap kasus apabila pelewatan pivoting menimbulkan ketakstabilan.

Keterbatasan. Sebagai catatan metodologis, matriks MNA dari rangkaian tangga sesungguhnya bersifat jarang (tridiagonal berbatas). Solver jarang atau algoritma Thomas dapat menyelesaikannya dalam orde mikrodetik, jauh lebih cepat daripada ketiga solver padat ini. Dengan kata lain, rangkaian tangga di sini berperan sebagai pembangkit matriks padat berukuran sembarang untuk menguji ketiga backend; angka percepatan yang dilaporkan menggambarkan kinerja *dekomposisi LU padat*—kernel LINPACK kanonik—dan bukan cara tercepat mensimulasikan rangkaian spesifik tersebut. Pemilihan algoritma yang tepat dapat mengungguli paralelisasi algoritma yang keliru.

VIII. Kesimpulan

Makalah ini memaparkan rancangan, implementasi, dan analisis sebuah solver rangkaian MNA yang menyelesaikan sistem linear $A\mathbf{x} = \mathbf{b}$ dengan tiga strategi: sekuensial, paralel OpenMP, dan paralel OpenCL. Kebenaran solver tervalidasi terhadap nilai analitik dan terhadap LTspice. Pengujian kinerja memperlihatkan fenomena *crossover* yang khas: paralelisasi merugikan untuk matriks kecil namun menguntungkan untuk matriks besar, dengan percepatan tertinggi $2{,}64\times$ (OpenMP 4 utas) dan $2{,}35\times$ (OpenCL) pada $n = 1001$. Analisis lebih lanjut mengungkap dua pelajaran penting: skalabilitas paralel dibatasi oleh *bandwidth* memori (sehingga utas berlebih dapat

TABLE II
WAKTU EKSEKUSI (MS) DAN PERCEPATAN TERHADAP SEKUENSIAL

n	t_{seq}	t_{omp4}	S_{omp2}	S_{omp4}	$S_{\text{omp,maks}}$	t_{ocl}	$S_{\text{ocl,komp}}$	$S_{\text{ocl,total}}$
51	0,027	1,569	0,02	0,02	0,01	2,316	0,01	0,01
101	0,189	3,113	0,10	0,06	0,02	4,345	0,04	0,04
201	1,705	6,923	0,31	0,25	0,10	11,390	0,15	0,15
301	5,253	11,693	0,56	0,45	0,19	16,216	0,32	0,32
501	26,684	24,166	1,03	1,10	0,55	33,102	0,81	0,78
701	72,265	44,505	1,38	1,62	0,97	56,838	1,27	1,22
1001	247,996	93,928	1,87	**2,64**	1,85	105,434	**2,35**	2,25

merugikan), dan akselerasi perangkat keras tidak menggantikan pemilihan algoritma yang tepat. Sebagai pengembangan, solver dapat diperluas dengan jalur algoritma Thomas/jarang sebagai pembanding, peluncuran kernel berkelompok untuk menekan *overhead* GPU, serta uji kasus matriks ber-*pivot* guna menguji ketahanan jalur GPU yang tanpa pivoting.

REFERENCES

[1] L. W. Nagel, "SPICE2: A computer program to simulate semiconductor circuits," Univ. California, Berkeley, Tech. Rep. ERL-M520, 1975.

[2] C.-W. Ho, A. E. Ruehli, and P. A. Brennan, "The modified nodal approach to network analysis," *IEEE Trans. Circuits Syst.*, vol. 22, no. 6, pp. 504–509, Jun. 1975.

[3] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Baltimore, MD, USA: Johns Hopkins Univ. Press, 2013.

[4] J. J. Dongarra, P. Luszczek, and A. Petitet, "The LINPACK benchmark: Past, present and future," *Concurrency Comput., Pract. Exper.*, vol. 15, no. 9, pp. 803–820, 2003.

[5] T. A. Davis and E. Palamadai Natarajan, "Algorithm 907: KLU, a direct sparse solver for circuit simulation problems," *ACM Trans. Math. Softw.*, vol. 37, no. 3, pp. 1–17, 2010.

[6] L. Dagum and R. Menon, "OpenMP: An industry standard API for shared-memory programming," *IEEE Comput. Sci. Eng.*, vol. 5, no. 1, pp. 46–55, Jan.–Mar. 1998.

[7] J. E. Stone, D. Gohara, and G. Shi, "OpenCL: A parallel programming standard for heterogeneous computing systems," *Comput. Sci. Eng.*, vol. 12, no. 3, pp. 66–73, May–Jun. 2010.

[8] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Comput. Conf.*, 1967, pp. 483–485.